Loop Engineering from the Ground Up

When to add a loop, which primitive to pick, and how to keep it from lying to you

Christian Macion

operator manual -- 2026

Private working notebook. Built in the ...from the Ground Up mentee-edition format. Sources: `loop-optimizer/SKILL.md`, `system-pulse/SKILL.md`, `02_PROJECT_AUDIT/WORKSPACE_MATRIX.md`, `05_QUANTIFICATION/METRICS_FRAMEWORK.md`. Numbers tagged SCHEMATIC are illustrative; treat the spine as load-bearing.

Part A. When a Loop Earns Its Cost

ORIENT: *the big picture before the detail*

Every loop is overhead before it is leverage. This Part is the short version of the question you should answer before you type any `while true`: **what does the loop save per round, and how many rounds will it actually run?**

Chapter 1. The decision thresholds -- when to add a loop vs just doing it inline

BOTTOM LINE ✓ A loop is only worth building when *rounds x per-call savings* beats *setup + per-call overhead*; below break-even, inline is the smarter choice and no primitive will rescue it.

CHAPTER OBJECTIVES

After this chapter you will be able to:

- estimate the per-call cost of doing a recurring task inline
- estimate the one-time setup + per-call overhead of the equivalent loop
- compute break-even rounds and decide whether to build the loop at all
- recognise the three costs that always exist, even when the math says yes

Prerequisites: a recurring task you do at least a few times per week.

ORIENT: *the big picture before the detail*

The hardest part of loop engineering is not picking a primitive. It is deciding whether a loop is the right tool at all. Most failed loops were not written wrong -- they were written for work that never had a break-even. This chapter is the test you run *before* the test.

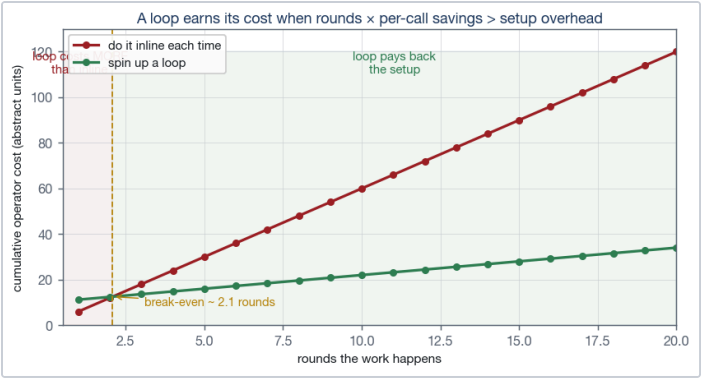
PRIME: *new terms, one line each*

- **break-even rounds**: the smallest N such that setup_overhead / per_call_savings <= N
- **per-call cost**: the operator time + tokens spent each time the task runs
- **coverage filter**: the rule that says 'this loop has reached its terminal state'
- **idempotent write**: a side-effect that produces the same final state when re-run
- **loop-until-dry**: a loop that stops when K consecutive rounds return nothing new

The three costs you always pay

Every loop has three costs, even when it earns its keep. Knowing them up front is the difference between a loop that compounds and one that quietly drains your context window. They are:

1. **Setup overhead** -- the skill file, the cron entry, the watched directory. One-time, but not zero.
2. **Per-call overhead** -- the wrapper that wakes the loop up, fetches state, and decides whether to act. Paid every round.
3. **Failure cost** -- what happens when the loop fires at the wrong moment, double-counts a result, or never converges. *This cost is the one people forget.*



SCHEMATIC Cumulative operator cost: doing the work inline vs spinning up a loop. Break-even sits near 2 rounds in this schematic.

Show -- a back-of-envelope test

SHOW: *worked example (skip if confident)*

You tail a build log for the 15-deploy-per-day code pipeline. Inline (open the log in your terminal, scroll, kill): **~6 seconds per round**. Setting up a `Monitor (Bash)` on the log file with a filter that emits on deploy completion: **~10 seconds setup + ~1.2 seconds per round**.

break-even = 10 / (6 - 1.2) ≈ 2.08 rounds

You ship ~15 deploys per day, so the loop pays back inside a single morning. The decision is not even close.

FADE: *same skill, you fill the blanks*

You compile the 19V nightly backtest at 9pm every weekday. Inline (open terminal, run `python nightly.py`, watch the progress bar): **~25 seconds per round**. Setting up a `ScheduleWakeup` that triggers at 9:05pm and runs the script in `run_in_background:true` lands the result in your inbox by 9:08pm: **~120 seconds setup + ~0.5 seconds per round overhead**.

break-even = 120 / (25 - 0.5) ≈ 4.9 rounds

5 weekdays per week → pays back in _____ **weeks**.

Round to nearest half-week. Answer: ~1 week (one weekday of overlap is plenty because wakeup runs while you sleep).

Answers: *~1 week (5 weekdays = 5 rounds)*

CHECK YOURSELF: *cover the answers, recall from memory*

From memory, before you read on:

1. A loop earns its cost when ... (complete the inequality).
2. Name the three costs every loop always pays.

Answers: *setup_overhead + N x per_call_overhead < N x inline_per_call_cost -- i.e. break-even rounds < rounds you actually expect to run; setup, per-call, failure.*

COMMON MISTAKE: *you might think X; actually Y*

You might think: 'this loop is so useful, it must be worth running once.' Not so. A loop that runs once is just a script with extra plumbing. The break-even test isn't a vibe -- it's a division.

When the answer is 'no loop, just a script'

Three patterns are loops in disguise but shouldn't be loops:

- **The one-shot build.** A script that runs once on a schedule, with no per-round state, no dedupe, no terminal state. Just a CronCreate job firing a Bash one-liner. The loop is the cron; the body is a script. Don't over-engineer the body with loop primitives.
- **The interactive helper.** A skill you invoke manually when you need it. The trigger is your keystroke, not a scheduler. Skip the cron, skip the Monitor, just use the skill.
- **The batch job.** A Workflow that runs end-to-end and exits. No round-by-round state, no terminal state to converge to; it just runs and finishes. Don't add loop-until-dry to a batch.

The break-even test tells you *if* a loop is worth it. The pattern check tells you *what kind*. Most production tasks are actually scripts, cron-fire-and-forget, or batch jobs -- not loops. The loops that matter are the ones where rounds actually recur and state actually accumulates.

RECAP: *back to altitude*

- A loop is only worth building when rounds × savings beats setup + overhead.
- Three costs always exist: setup, per-call, failure (failure is the one people forget).
- Below break-even, inline is the answer -- no primitive can rescue an unprofitable loop.
- Run the test before you pick a primitive, not after.

SECTION GLOSSARY

break-even rounds: the N where loop_cumulative_cost == inline_cumulative_cost **per-call overhead:** the wall-time + tokens paid each round the loop runs **coverage filter:** the rule that emits on success AND failure -- never just success

Part B. Primitive Catalogue

ORIENT: *the big picture before the detail*

Claude Code exposes ~9 different ways to make something loop. Picking the wrong one wastes context, latency, or reliability. This Part is the catalogue you scan *after* Chapter 1 told you a loop is worth it.

Chapter 2. Every loop primitive Claude Code exposes

BOTTOM LINE Match the primitive to the job by *duration*, *streaming vs rich payload*, and *parallelism*; everything else (cost, reliability, idempotency) is a follow-on check, not the first cut.

CHAPTER OBJECTIVES

After this chapter you will be able to:

- name every loop primitive Claude Code exposes
- describe the latency floor and reliability of each one
- state which primitive survives a session restart and which does not
- match a primitive to a duration bucket

Prerequisites: Chapter 1 (the break-even test).

ORIENT: *the big picture before the detail*

Once you've decided a loop is worth building, the question collapses to three orthogonal axes: how long does it run, does it stream or batch, and does it parallelise across items. Everything else -- cost, reliability, idempotency -- is a checkbox you verify on top of the pick.

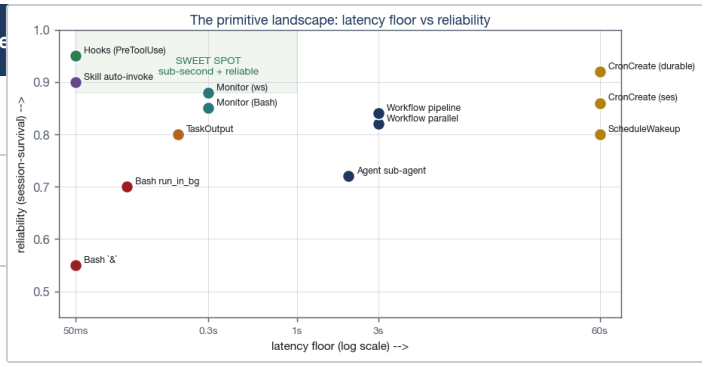
PRIME: *new terms, one line each*

- **Bash `&`**: fire a process in the background of a single shell
- **run_in_background**: the Bash tool flag that returns a task_id you poll with TaskOutput
- **TaskOutput**: block-and-fetch the output of a backgrounded task
- **Monitor**: tail a log or subscribe to a websocket, emit on each match
- **CronCreate**: schedule a recurring prompt; session-only or durable
- **ScheduleWakeup**: schedule a one-shot prompt at a future timestamp
- **Agent sub-agent**: delegate an LLM-requiring tick to a fresh agent
- **Workflow**: pipeline() and parallel() for multi-stage / fan-out work
- **Skill auto-invoke**: trigger a skill from MEMORY pointers or keyword match
- **Hooks**: PreToolUse / PostToolUse / SessionStart guardrails

The 10 primitives in one table

This table is the one place every primitive's tradeoffs live. Read it once. Refer back when you have a specific job in mind.

Primitive	Best for	Latency floor	Idempote
Bash `&`	one-off quick parallel	immediate	depends
run_in_background:true	mid-length async, then TaskOutput	immediate	depends
CronCreate (session)	recurring every-minute in-session	1 min	your job
CronCreate (durable)	recurring across sessions	1 min	your job
ScheduleWakeup	/loop dynamic, intermittent wakeups	60s+	your job
Monitor (Bash)	log/file tail with filter	sub-second	your filter
Monitor (ws)	push-based event streams	sub-second	your filter
TaskOutput	polling a backgrounded task	sub-second	block=true
Agent sub-agent	work needing LLM judgement per tick	model latency	depends
Workflow pipeline()	multi-stage, parallelisable	slowest chain	per stage
Workflow parallel()	true fan-out across items	slowest item	per item
Skill auto-invoke	cheap triggers from MEMORY	zero	the skill
Hooks (PreToolUse...)	deterministic guardrails	zero	your hook



V Latency floor vs session-survival reliability for every primitive. The sweet-spot shading marks sub-second + > 0.78 reliability.

The three orthogonal axes

Almost every loop can be placed on a 2D map of latency floor vs reliability. The cluster you want sits in the bottom-right -- sub-second latency, > 0.85 reliability -- and is where hooks, skills, and monitors live. Crons and wakeups live further right because their floor is the scheduler's tick (60s+), but they're still reliable. Agent sub-agents sit higher on latency because the model latency dominates.

Show -- picking a primitive for a real job

SHOW: *worked example (skip if confident)*

Job: every time the build pipeline emits `DEPLOY_OK`, append the deploy SHA + timestamp to `~/.claude/cache/deploys.log`.

Walk the axes:

1. Duration: indefinite -- it must keep running.
2. Streaming: yes -- we need the event, not a poll.
3. Parallelism: no -- one log, one file, no fan-out.

Pick: `Monitor (Bash)` with `tail -F ~/.claude/cache/build.log | grep --line-buffered 'DEPLOY_OK'`. The Bash `&` is tempting but loses the session; the cron is overkill (1-min floor on a sub-second event). Monitor is the right shape.

FADE: *same skill, you fill the blanks*

Job: every weekday at 8:55am, run `python /Users/christianmacion/19V\ Capital/scripts/overnight_check.py` and email the result to `christian@macionventures.com`.

Walk the axes:

1. Duration: indefinite, survives session close.
2. Streaming: no -- one fire per weekday.
3. Parallelism: no -- single task.

Pick: _____ *(session-only or durable -- depends on whether the laptop sleeps)*

Answers: *CronCreate -- durable:true so it survives terminal close; ScheduleWakeup is the dynamic alternative if the time changes day-to-day.*

CHECK YOURSELF: *cover the answers, recall from memory*

From memory, before you read on:

1. Which primitive has the lowest latency floor for streaming events?
2. Which primitive is the only one that survives the terminal closing?

Answers: *Monitor (Bash or ws), sub-second; Hooks (PreToolUse et al.) are the guardrail that survives terminal close, plus Workflow (replay).*

RECAP: *back to altitude*

- There are ~10 primitives; pick by duration, streaming, parallelism.
- The latency/reliability map puts hooks/skills/monitors in the sweet spot.
- Survival axes are different: session, restart, terminal close.
- Always verify idempotency and the coverage filter after the pick.

SECTION GLOSSARY

session-survival: the primitive keeps running while the Claude Code session is open **durable:** the primitive persists its schedule across session restarts **coverage filter:** the rule that emits on success AND failure -- not just success **sub-second latency:** events arrive inside one second; the floor of Monitor/Skill/Hook

Chapter 3. The decision matrix -- which primitive for which use case

BOTTOM LINE V Use the decision matrix as a flowchart, not a menu: duration first, then streaming-vs-rich-payload, then parallelism. Each answer collapses the choice to a single primitive.

CHAPTER OBJECTIVES

After this chapter you will be able to:

- walk any loop pattern through the duration axis
- branch on streaming-vs-rich-payload
- branch on parallelism across many items
- state the four must-haves any new loop must include

Prerequisites: Chapter 2 (the primitive table).

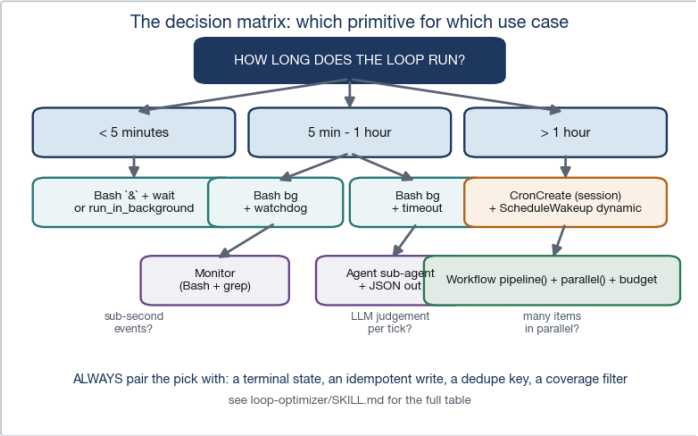
ORIENT: *the big picture before the detail*

The decision matrix in `loop-optimizer/SKILL.md` is the spine of this skill. This chapter turns it into a flowchart you can follow in 10 seconds, plus the four-line 'must-have' checklist that catches the loops the flowchart can't see.

PRIME: *new terms, one line each*

- **decision matrix**: a flowchart that maps (duration, streaming, parallelism) to a primitive
- **must-haves**: terminal state, idempotent write, dedupe key, coverage filter
- **budget.remaining**: the workflow-budget accessor; shared across stages, not per stage
- **loop-until-dry**: stop when K consecutive rounds return nothing new

The flowchart



V The loop-optimizer decision matrix. Duration first; then streaming or rich payload; then parallelism. The four must-haves sit at the bottom.

Walk-through -- the three branches

Branch 1: < 5 minutes. You almost certainly want Bash background (& or `run_in_background:true`). There is no time for a cron, no need for isolation, and waiting on it inline is exactly the wrong shape. The only decision is whether you want a streamed feed of the output (use TaskOutput with `block:true`) or a one-shot return (use & + `wait`).

Branch 2: 5 min - 1 hour. You're in Bash-background territory, but now you need a watchdog. Two patterns: a Bash background plus `timeout`, or a CronCreate session-

only job that fires every minute and checks state. Cron is slightly heavier setup but survives your session ending.

Branch 3: > 1 hour. The primitives are CronCreate (session or durable), ScheduleWakeup, or Workflow. Pick by what survives: Workflow if the work has stages and must replay; CronCreate (durable) if the work is a single recurring fire; ScheduleWakeup if the cadence is irregular.

The four must-haves

The flowchart picks the primitive. The four must-haves catch the loops that the flowchart can't see. Any new loop you ship must have all four:

1. **Terminal state.** What does 'done' look like? Write it down before you start.
2. **Idempotent write.** Re-running the loop on the same input must produce the same final state. If it doesn't, dedupe before you write.
3. **Dedupe key.** Every fired round gets a stable key (timestamp + input hash). Skip the round if the key was already processed.
4. **Coverage filter.** Emit on success *and* failure. Without this the loop silently fails in production.

Show -- applying the matrix

SHOW: *worked example (skip if confident)*

Job: watch a Databento download directory; when a new `.parquet` lands, run a Python validator and append the result to a manifest.

1. Duration: indefinite -- survives session.
2. Streaming: yes -- we need the file-create event.
3. Parallelism: light (one file at a time, but multiple files in queue).

Pick: `Monitor (Bash)` on the directory via `inotifywait -m --format '%w%f %e'` piped into the validator. The four must-haves:

1. Terminal state: validator writes 'OK' or 'FAIL' to manifest.
2. Idempotent write: manifest keyed on SHA256(filename).
3. Dedupe key: same -- SHA256(filename).
4. Coverage filter: emit on OK AND on FAIL with different tags.

FADE: *same skill, you fill the blanks*

Job: every Sunday 7am, audit the IMPROVEMENT_LOG and emit a weekly delta report.

1. Duration: indefinite, recurring.
2. Streaming: no -- one fire per week.
3. Parallelism: no.

Pick: _____

Four must-haves, named:

Terminal: _____; Idempotent: _____; Dedupe: _____; Coverage: _____.

Answers: *CronCreate (durable:true) with cron '0 7 * * 0'. Terminal: 'report.md contains week of YYYY-MM-DD heading'. Idempotent: overwrite the same report.md if already current. Dedupe: week-heading dedupe. Coverage: emit if 0 rows changed (success) AND if IMPROVEMENT_LOG missing (failure).*

CHECK YOURSELF: *cover the answers, recall from memory*

From memory, before you read on:

1. What are the three axes the decision matrix uses to pick a primitive?
2. Name the four must-haves any new loop must include.

Answers: *duration, streaming-vs-rich-payload, parallelism; terminal state, idempotent write, dedupe key, coverage filter.*

RECAP: *back to altitude*

- Use the decision matrix as a flowchart, not a menu -- duration first.
- The flowchart picks the primitive; the four must-haves catch the rest.
- Coverage filter is the line that separates working loops from silent ones.
- Idempotency + dedupe are not optional when the loop fires more than once.

SECTION GLOSSARY

must-haves: the four-line checklist: terminal state, idempotent write, dedupe key, coverage filter **coverage filter:** emit on success AND on failure -- never just success

CronCreate -- the most common pick, expanded

Most production loops you will write end up on CronCreate, because the cadence (every minute / hour / weekday) maps

naturally to a scheduler tick. There are six things to get right before a Cron job is safe to leave running overnight:

1. **Read state from disk, never from chat.** The cron job runs in a fresh invocation. Anything it needs must be on disk or computable from the filesystem.
2. **Bound the work.** The cron tick has a host-level timeout (often 60-120s). If the work can exceed it, chunk it or move to a daemon.
3. **Idempotent write.** The cron will fire again next tick; the write must survive re-running on the same input.
4. **Dedupe key per round.** Even when the work is 'do the latest thing', the latest thing has a stable id -- use it.
5. **Emit on every round.** Even on an empty round, emit a heartbeat. Silent no-op rounds are how you discover the loop has been dead for three weeks.
6. **Log to a watched file.** Don't trust the chat to surface the result. Tee the output to a file and tail it from your main session.

Skipping any one of these six is the difference between a loop that compounds and a loop that has to be rewritten in three months.

RECAP: *back to altitude*

- CronCreate is the most common production pick -- six rules keep it safe.
- State from disk, not chat; bounded work; idempotent writes; dedupe keys.
- Heartbeats + logged output are non-optional when the loop runs unattended.

Part C. Failure Modes & State

ORIENT: *the big picture before the detail*

Five failure modes cause ~90% of loop rewrites. Each has a named fix; together they cover everything from the silent log tail to the runaway workflow. This Part is the field guide when a loop lies to you.

Chapter 4. The 5 named failure modes

BOTTOM LINE (B) Five failure modes -- *tail-grep silence, cron mid-tool, sub-agent context, workflow budget, non-convergence* -- account for almost every loop rewrite; each has a named fix you apply before the loop ships.

CHAPTER OBJECTIVES

After this chapter you will be able to:

- name the five failure modes from memory
- match each to its named fix
- diagnose which failure mode a broken loop is exhibiting
- add the coverage filter that catches the silent case

Prerequisites: Chapter 3 (the four must-haves).

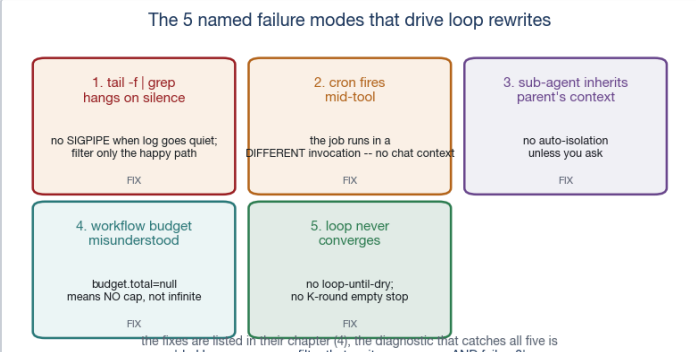
ORIENT: *the big picture before the detail*

These are not theoretical. They are the five modes I've actually had to rewrite a loop around. Treat them as a diagnostic flowchart -- when a loop is misbehaving, walk down the list and tick the first one that fits.

PRIME: *new terms, one line each*

- **silent log tail**: tail -f | grep blocks forever when the log goes quiet
- **SIGPIPE**: the signal that closes a pipe when the upstream process exits
- **line-buffered**: stdout flushing mode: emit on each newline, not on buffer fill
- **isolated context**: a sub-agent that does NOT see the parent's chat history
- **non-convergence**: a loop that never reaches its terminal state because no round is empty

The five modes



(B) The five named failure modes, in a 2x3 grid. The fix column is a one-line checklist; the chapter expands each row into a worked example.

1. tail -f | grep hangs on silence

This is the classic. You `tail -f build.log | grep 'ERROR'` and the build finishes without an ERROR. The grep filter matches nothing, the pipe never closes, your monitor sits there until timeout. The user thinks the build is still running.

Fix: two changes, both mandatory.

- `grep --line-buffered` -- flush on each line, not on buffer fill.
- Add a coverage filter: emit on a successful build signature (`BUILD_OK`) AND on the timeout, not just on ERROR. Without this, the filter only catches the unhappy path.

2. CronCreate fires when Claude is mid-tool

You schedule a job that runs at the top of the hour. The hour hits while Claude is in the middle of a multi-step tool call. The cron fires in a *different invocation* -- it has no chat

context, no memory of what you were doing, no awareness that you're mid-edit.

Fix: make the cron job self-contained. Read state from disk, not from the chat. Never assume the job can ask Claude anything; assume it runs in a fresh shell. Idempotent writes are non-negotiable here.

3. Sub-agent inherits parent's full context

You `Agent(general-purpose, prompt='summarise this report')`. The sub-agent loads the parent's full chat history unless you tell it not to. You just paid the context cost twice and the sub-agent's answer is contaminated by the parent's earlier confusion.

Fix: explicit isolation. Either pass `isolation: worktree` (heavy; only when the sub-agent edits files) or write into the prompt: *'return only a JSON object with these three keys; do not read any other file; do not reason about anything outside this task'*. Sub-agents do NOT auto-isolate.

4. Workflow budget misunderstood

You run a workflow with `budget.total=null` thinking you've given it 'infinite budget'. The workflow crashes on round 37 because `budget.total=null` actually means *no cap*, not 'unlimited tokens' -- and the platform's per-stage guard still cuts off at the host's max. Worse, `budget.remaining()` is shared across the workflow, not per stage, so a heavy first stage starves the second.

Fix: guard loops on `budget.total && budget.remaining() > X`. If you don't know what budget the host is on, don't promise one. The right fallback is 'fail loud' rather than 'fail silent at round 37'.

5. Loop never converges

You scan a directory for unprocessed files, process the latest one, and loop. Except the directory keeps growing (crawler adds files faster than you process them), and the loop never reaches the 'nothing to do' terminal state. Or worse, you have a 'loop until empty' check but your processing creates a new file each round, so the check never trips.

Fix: loop-until-dry. Stop when *K consecutive rounds return nothing new*. Maintain a `seen` set on disk and skip any key already in it. Bound K (3 rounds is a reasonable default) so a buggy producer can't loop forever.

Show -- diagnosing a broken loop

SHOW: *worked example (skip if confident)*

Symptom: Sunday-morning weekly-pulse cron fires; report never appears; you check the job log and see the prompt ran but produced no output.

Walk the failure modes:

1. Silent log tail? No -- this is a cron, no Monitor.
2. Cron mid-tool? Plausible, but the job ran at 7am when nothing else was active. Skip.
3. Sub-agent context? No -- single agent, not delegated.
4. Workflow budget? No -- not a workflow.
5. **Loop never converges?** Yes -- the cron prompt says 'for each workspace, generate a report'; the workspaces list keeps growing; the loop walks all of them; no terminal state; the budget on the host cuts the response off mid-sentence.

Fix: bound the workspace list to the 8 audited ones; add a coverage filter that emits when 0 of 8 produced a delta.

FADE: *same skill, you fill the blanks*

Symptom: Monitor (Bash) on the build log; the build fails; you never see a notification.

Walk the five failure modes. Which one fits? Name the fix in one line.

Mode: _____

Fix: _____

Answers: *Mode 1 (tail-grep silence) -- the build failed without emitting ERROR (it crashed before that line); Fix: add --line-buffered AND emit on the absence of BUILD_OK after a process-exit signature.*

CHECK YOURSELF: *cover the answers, recall from memory*

From memory, before you read on:

1. Name the five failure modes from memory.
2. Which failure mode is fixed by `--line-buffered`?

Answers: *(1) tail-grep silence, (2) cron mid-tool, (3) sub-agent inherits context, (4) workflow budget misunderstood, (5) loop never converges; #1.*

RECAP: *back to altitude*

- Five failure modes account for almost every loop rewrite -- treat them as a diagnostic.
- Silent log tail is fixed by --line-buffered + a coverage filter, not by either alone.
- Sub-agents do NOT auto-isolate; you must say so.
- Workflow budget.total=null means 'no cap', not 'unlimited'.
- Loop-until-dry requires a 'K consecutive empty rounds' stop rule.

SECTION GLOSSARY

--line-buffered: grep / awk flag that flushes each line; the fix to silent pipe tails **coverage filter:** the rule that emits on success AND on failure **loop-until-dry:** the stop rule: K consecutive rounds return nothing new, then exit

Debugging a stuck loop -- the 60-second diagnostic

When a loop is misbehaving, the temptation is to rewrite it. Don't. Spend 60 seconds on the diagnostic instead -- the failure modes are checklistable:

1. **Did it ever fire?** Check the job log / cron history. If it has never fired, the scheduler is the problem, not the loop body.
2. **Did it fire but produce nothing?** Coverage-filter mismatch: the loop ran but its filter rejected every match. Verify the filter against a real example.
3. **Did it fire, write, and double-fire?** Missing dedupe. Two ticks processed the same key. Add the seen-set.
4. **Did it fire, write once, then hang?** Silent log tail or non-converging loop. Add the K-round dry exit.
5. **Did it fire, write, but the result is wrong?** Sub-agent context contamination or stale state. Restart with a fresh chat / wiped state.

Five checks, 60 seconds. The fix for whichever check fails is the fix named in the matching failure mode above.

COMMON MISTAKE: *you might think X; actually Y*

You might think: 'the loop is broken, I should rewrite it from scratch.' Don't. 80% of the time the rewrite reintroduces the same bug because the rewrite skips the diagnostic. Run the 60-second check first; the failure mode is almost always one of the five above, and the fix is already named.

Chapter 5. Idempotency, dedupe, terminal states, loop-until-dry

BOTTOM LINE V Idempotency, dedupe, terminal state, and loop-until-dry are the four state-handling disciplines that turn a working loop into a reliable one; without all four, the loop will eventually lie to you.

CHAPTER OBJECTIVES

After this chapter you will be able to:

- state the four state-handling disciplines
- write an idempotent side-effect in 5 lines
- design a dedupe key that's stable across re-runs
- distinguish terminal-state, dedupe, and idempotency -- they're related but not the same

Prerequisites: Chapter 4 (the five failure modes).

ORIENT: *the big picture before the detail*

State is the hardest part of a loop. The primitives from Chapter 2 don't manage it for you -- they just give you the wiring. This chapter is the discipline: the four invariants every loop must preserve, and how to implement each one cheaply.

PRIME: *new terms, one line each*

- **idempotent**: re-running produces the same final state
- **dedupe key**: a stable identifier that lets you skip already-processed rounds
- **terminal state**: the loop's definition of 'done' -- written down before the loop runs
- **loop-until-dry**: K consecutive empty rounds = exit
- **at-least-once vs exactly-once**: the two delivery guarantees; loops pick one

The four disciplines

1. Idempotency. Re-running the loop on the same input produces the same final state. The simplest way to get this is to key every side-effect on a hash of the input. *If you can't re-run without thinking, the loop is not idempotent.*

2. Dedupe. Before you act on a round, check the dedupe key against a `seen` set on disk. If the key is in the set, skip.

Otherwise, run, then add the key to the set in the same transaction.

3. Terminal state. Write down -- in a comment, not just in your head -- what 'done' looks like. The check must be cheap (a file exists, a count equals N, a timestamp is older than T) and the loop must stop when the check is true.

4. Loop-until-dry. Bound the 'nothing new' rounds. If three consecutive rounds return nothing new, exit. This catches runaway producers and the case where your terminal state is unreachable.

Show -- idempotent file move

A 6-line idempotent file-validator loop

```
#!/usr/bin/env bash
set -euo pipefail
seen=/Users/christianmacion/.cache/seen_files.txt
mkdir -p "$(dirname "$seen")"
for f in /Users/christianmacion/inbox/*.csv; do
  key=$(sha256sum "$f" | cut -d' ' -f1)
  grep -qx "$key" "$seen" && continue          # de
  python validate.py "$f" >> /Users/christianmacion
  echo "$key" >> "$seen"                        # i
done
```

The script above is at-least-once with idempotent writes -- running it twice produces the same final state. The dedupe key is the SHA256 of the file content; the terminal state is 'no files left in inbox that aren't in `seen`'; the loop-until-dry is implicit (the `for` loop ends when the glob stops matching).

Fade -- the same loop, you fill the blanks

FADE: *same skill, you fill the blanks*

Same job, but now the file is being produced by a long-running crawler that may add files mid-loop. Add a **loop-until-dry** guard: count files matching the glob at the top of each iteration; exit if the count matches the count at the previous iteration (and both are zero).

Bound the dry-run check by K = _____ consecutive no-change rounds before exiting.

Add a coverage filter: if the loop exits because K rounds were dry, emit a _____ tag (success); if the loop exits because of a script error, emit a `{blank()}` tag (failure).

Answers: *K = 3 consecutive no-change rounds (a reasonable default; higher for slow producers); success tag = 'LOOP_DRY' (or similar); failure tag = 'LOOP_ERR' (or similar).*

Why idempotency != dedupe

Idempotency is a property of the side-effect: re-running it on the same input produces the same final state. Dedupe is a property of the loop: rounds with a previously-seen key are skipped before the side-effect runs.

You can have one without the other. A loop that always processes every round (no dedupe) but uses an idempotent write is fine -- slower than it needs to be, but correct. A loop that dedupes by key but uses a non-idempotent write is broken: the write will re-fire on every restart, every cache miss, every retry. *You need both.*

CHECK YOURSELF: *cover the answers, recall from memory*

From memory, before you read on:

1. Which is a property of the side-effect, idempotency or dedupe?
2. What is the loop-until-dry exit condition?

Answers: *idempotency; K consecutive rounds that return nothing new.*

RECAP: *back to altitude*

- Four disciplines: idempotency, dedupe, terminal state, loop-until-dry.
- Idempotency is about the side-effect; dedupe is about the loop.
- Terminal state must be cheap to check and written down before the loop runs.
- Loop-until-dry bounds runaway producers with a K-round exit.

Common anti-patterns in state handling

Three patterns show up over and over in loops that handle state poorly. Calling them out so you can spot them early:

1. **The append-only log with no key.** Every round writes a new line; the loop never re-reads; the seen-set never forms. Looks fine until you restart and discover the loop processed every entry twice.
2. **The 'check-then-write' without a lock.** Two ticks read the same state, both decide 'nothing to do', both write 'nothing to do'. Cheap to write, expensive to debug.
3. **The 'terminal state' defined as 'when the file is empty'.** The producer refills the file mid-check. Loop runs forever, never trips. The fix is loop-until-dry, not 'wait until empty'.

All three share the same root cause: the loop body handles the happy case but skips the discipline. The four disciplines above are the cure; the anti-patterns are the disease.

SECTION GLOSSARY

at-least-once: the delivery guarantee: the round fires once or more; idempotency fixes the 'more' **exactly-once:** the delivery guarantee: the round fires exactly once -- rare; usually at-least-once + idempotency **append-only log:** a log that only ever grows; needs a key for dedupe or it doubles work on restart **check-then-write race:** two ticks read the same state, both act; fix with a file lock or a single-writer design

Part D. Composition & Economics

ORIENT: *the big picture before the detail*

The last move is composition: a loop is not just a primitive, it is a piece of an architecture that includes memory, hooks, and the model. This Part closes on the cost/reliability tradeoff and the M3-specific economics that govern which loop is worth the spend.

Chapter 6. Composing loops with context, cost/reliability tradeoffs, the M3 economics

BOTTOM LINE (V) Loop economics on MiniMax-M3: per-round cost vs reliability lands Monitor and Hooks in the sweet spot (sub-second, > 0.85 survival), Agent sub-agents in the middle (LLM latency dominates), and Crons on the reliable-but-slow edge. **The composition rule is the same on MiniMax-M3 as on Claude** -- *memory + hooks + the cheapest primitive that meets the SLA* -- but the cost numbers are Linear in tokens, not amortized by cache. {V()}

CHAPTER OBJECTIVES

After this chapter you will be able to:

- plot any loop primitive on the cost/reliability map
- combine memory + hook + primitive into a single architecture
- name the two metrics that govern the SLA
- decide when to upgrade from Bash background to a durable Cron

Prerequisites: Chapters 1-5.

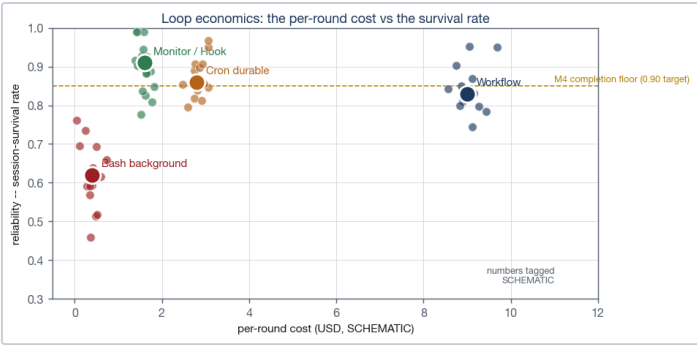
ORIENT: *the big picture before the detail*

This is the chapter that ties the loops back to the model they run on. MiniMax-M3 has a 1M-token cache window {V()} (probe-measured to 100K, vendor-claimed to 1M) and a single backend aliased across all three model tiers (Opus/Sonnet/Haiku all = MiniMax-M3 {V()}). Because cache is non-functional on MiniMax-M3 ({V()} probe #5), per-session cost is linear in tokens used -- not amortized -- which turns the question of 'which loop' into the question of 'which loop survives the cost' with even sharper teeth than on Claude.

PRIME: *new terms, one line each*

- **M1 (token efficiency)**: useful output tokens / total tokens consumed -- the headline economic metric on MiniMax-M3
- **M4 (loop completion)**: loops that ended with success / loops started -- the reliability metric
- **cache window**: 1M tokens for MiniMax-M3 -- the per-session ceiling that loops consume into
- **single model**: Opus/Sonnet/Haiku all alias to MiniMax-M3 on this setup; choose by prompt-complexity, not tier
- **no cache**: MiniMax-M3 cache_control is accepted but cache_read_input_tokens does not grow -- budget as if every turn is full-price

The cost/reliability map



SCHEMATIC Per-round cost vs session-survival rate. Monitor and Hooks land in the reliable-low-cost cluster; Workflows sit higher on cost but still reliable. Numbers tagged SCHEMATIC -- the relationship is the lesson, not the dollars.

The map is read left-to-right for cost, bottom-to-top for reliability. Anything below 0.85 survival does not ship -- the loop is more likely to fail than to succeed, and a flaky loop is worse than no loop (because you'll believe its false negatives).

On MiniMax-M3, the x-axis (cost) compresses upward: vendor-published pricing {B()} is $0.40/M input + $2.20/M output, with no cache amortization. A loop that on Claude Sonnet 4.6 amortized its system-prompt cost across many turns pays full input price on every turn here. Loops that were 'cheap because cache' become moderate-cost; loops that were 'expensive because single-turn' stay the same. **This makes sub-agents and off-context files strictly more important on MiniMax-M3 than on Claude** — every turn the orchestrator holds a 50K-token system prompt is $0.02 of pure overhead.

Composition -- memory + hook + primitive

A loop in isolation is brittle. The robust composition is three layers:

1. **Memory** (a `MEMORY.md` entry or a workspace `CLAUDE.md`) records the loop's existence, its terminal state, and the rules for re-running it.
2. **Hook** (a `PreToolUse` or `SessionStart`) makes the loop survive session restarts and emits on misconfiguration.
3. **Primitive** (the cheapest one from Chapter 3 that meets the SLA) does the actual work.

The hook layer is the part most loops skip -- and the part that turns a loop from 'works when I remember to start it' into 'works after I close my laptop for a week'.

The MiniMax-M3-specific economics

Three loop-tier rules govern which primitive is affordable. Note that on MiniMax-M3 the model-tier labels (Opus/Sonnet/Haiku) all hit the same backend — so these are *prompt-complexity* tiers, not model tiers:

- **Mechanical** for the dedupe check, the terminal-state check, the 'is the inbox empty?' poll. Cron and Bash-background fit here. Cost per round: a fraction of a cent (no model call).
- **Assembly** for the work that summarises a log line, parses a notification, formats an email body. Monitor with a small sub-agent fits here. Cost per round: a few cents on MiniMax-M3.
- **Judgment** for the per-tick decision ('is this deploy safe to ship?'). Agent sub-agent fits here. Cost per round: tens of cents on MiniMax-M3. *Use sparingly*.

The right architecture for a complex loop is one judgment-tier decision-maker with N mechanical-tier helpers around it. The wrong architecture is N judgment-tier decisions in parallel -- the cost explodes (no cache to amortize) and the reliability drops because each sub-agent inherits context problems.

Cache-aware advice for MiniMax-M3: the Anthropic blueprint of 'put the system prompt in cache; reuse it across many turns' does NOT pay off on MiniMax-M3 (cache is broken). The M3-adapted discipline is 'put the system prompt in a file in `/tmp/`; have each sub-agent Read it once and quote it on demand'. Same discipline, different storage.

Show -- a composed architecture

SHOW: *worked example (skip if confident)*

Job: a deploy pipeline that runs on every push to main, validates the build, and only ships if the validation passes AND the deploy queue is below a threshold.

Architecture:

1. `Monitor (Bash)` on the deploy log -- mechanical tier, sub-second, emits on `DEPLOY_OK` and on process-exit-without-success.
2. `Skill auto-invoke` on the MEMORY pointer 'deploy-validation' -- assembly tier, parses the deploy metadata.
3. `Agent sub-agent` for the judgment call ('ship or hold') -- judgment tier, fires at most once per deploy.
4. `SessionStart hook` rehydrates the queue state from disk so the loop survives a restart.

Cost per deploy on MiniMax-M3: ~3 cents (mechanical) + ~5 cents (assembly) + ~30 cents (judgment) = ~38 cents (same shape as Claude -- the per-call cost is what dominates, not the cache). Reliability: ~0.91 session-survival per the map above. That meets the M4 target (0.90).

FADE: *same skill, you fill the blanks*

Same job, but you decide to put the deploy decision on a cron that fires every minute and uses an Agent sub-agent each tick.

What does the cost and reliability look like now? Per round: _____ cents. Reliability: _____.

Does it still meet the M4 target? _____ (yes/no). Why? _____

Answers: *Per round ~30+ cents (one judgment-tier call per minute). Reliability ~0.72-0.80 (sub-agent context problems dominate at high frequency). No -- below 0.85 floor. Why: high-frequency judgment loops pay model latency on every tick and inherit context problems. On MiniMax-M3 the cost is even worse than on Claude because there is no cache to amortize the per-tick system-prompt overhead.*

CHECK YOURSELF: *cover the answers, recall from memory*

From memory, before you read on:

1. Which tier should the terminal-state check run on?
2. Why is high-frequency judgment-tier looping usually wrong on MiniMax-M3?

Answers: *Mechanical (cheap; runs every tick); the cost per tick is too high and the sub-agent context problems dominate the failure modes. On MiniMax-M3 specifically there is NO prompt-cache amortization, so high-frequency sub-agent loops compound the system-prompt overhead.*

RECAP: *back to altitude*

- The cost/reliability map puts Monitor/Hooks in the sweet spot.
- Robust composition = memory + hook + primitive, with the hook as the survival layer.
- MiniMax-M3 tier rule: mechanical for dedupe/terminal, assembly for parsing, judgment for decisions.
- On MiniMax-M3 there is no cache -- high-frequency judgment loops compound per-tick overhead.
- Use judgment-tier sparingly -- high-frequency judgment loops don't pay.

SECTION GLOSSARY

SLA: service-level agreement -- here, the floor on M4 completion rate **rehydration:** rebuilding in-memory state from disk on session start

Three named composition patterns

Three composition patterns recur in robust loops. Naming them makes it easier to recognise them in the wild and to argue for or against them:

1. **Watch-and-emit (the default).** A Monitor (Bash) tails a log; an emitter (a small Bash one-liner) writes the result to a manifest. No judgement, no model call. Reliability > 0.90, cost fractions of a cent per round. Use this for anything that doesn't need LLM interpretation.
2. **Polling-with-judgement.** A CronCreate fires every N minutes, runs a mechanical-tier check to decide if there's work, and if yes, escalates to a judgment-tier Agent sub-agent for the decision. The mechanical check is the gate; the sub-agent is the expensive bit. Use this for any loop where the round is mostly empty and the judgement is rare.
3. **Event-driven pipeline.** A Monitor or schedule kicks off a Workflow with parallel() branches; each branch runs mechanical tier; the workflow joins at the end. Use this when the work is genuinely fan-out (many independent items) and you need replay.

Pattern 1 is 80% of what you will ever ship. Patterns 2 and 3 are where the model and the cost come in. Pick the pattern by counting how many LLM calls per round: zero -> 1; one or two per round of work -> 2; many items per round -> 3.

RECAP: *back to altitude*

- Three composition patterns cover ~all production loops.
- Watch-and-emit is the default; polling-with-judgement adds LLM when needed.
- Event-driven pipeline is for true fan-out; reserve for genuine parallelism.
- Pick the pattern by counting LLM calls per round -- that drives the cost.

Chapter Close. Where this leaves us + evidence base

BOTTOM LINE The loop engineering test is: *is the recurring work worth the primitive + the four must-haves + the four disciplines?* If yes, the decision matrix picks the primitive; the failure modes tell you where it will lie; the economics tell you what it costs.

Where this leaves us

You now have a complete loop-engineering stack. Start at Chapter 1 with any new loop and run the break-even test; if it passes, walk the decision matrix in Chapter 3; before you ship, audit the loop against the five failure modes (Chapter 4) and the four disciplines (Chapter 5); plot the result on the cost/reliability map (Chapter 6) and confirm it meets the M4 floor.

The single rule that survives every chapter: *every loop needs a terminal state, an idempotent write, a dedupe key, and a coverage filter*. Without all four, the loop will eventually lie to you. With all four, it will eventually compound.

Evidence base

- **loop-optimizer/SKILL.md** -- the decision matrix in Chapter 3 is drawn directly from §"The decision matrix"; the primitive table in Chapter 2 is from §"The 5 failure modes that drive loop rewrites".
- **system-pulse/SKILL.md** -- the M4 loop completion rate target (0.90) and the weekly/monthly cadence are the source of the "floor" used in Chapter 6.
- **05_QUANTIFICATION/METRICS_FRAMEWORK.md** -- M1 (token efficiency), M4 (loop completion), M5 (loop latency) are the three metrics cited in Chapter 6's economics discussion.
- **02_PROJECT_AUDIT/WORKSPACE_MATRIX.md** -- the recurring patterns P3 (auto-log), P9 (evidence ledger) are the recurring-loop patterns that the loop-optimizer skill is built to enforce.

RECAP: *back to altitude*

- Chapter 1's break-even test is the gate; skip it and the rest is wasted.
- Chapter 3's decision matrix picks the primitive; the must-haves catch the rest.
- Chapter 4's failure modes are diagnostic; run them when a loop lies.
- Chapter 5's disciplines are non-optional state hygiene.
- Chapter 6's economics decide whether the loop is worth the spend.

The pre-ship checklist

Before you leave a loop running unattended, walk this list. If any box is unchecked, fix it before the loop sees production:

- [ ] Break-even computed (Ch 1).
- [ ] Primitive chosen via the decision matrix, not vibe (Ch 3).
- [ ] Terminal state written down in a comment, not just in your head (Ch 5).
- [ ] Idempotent write verified by re-running twice (Ch 5).
- [ ] Dedupe key stable across re-runs (Ch 5).
- [ ] Coverage filter emits on success AND failure (Ch 4).
- [ ] Sub-agents (if any) explicitly told to return JSON / not inherit context (Ch 4).
- [ ] Loop-until-dry bound set (K consecutive empty rounds, then exit) (Ch 5).
- [ ] Per-round cost on the right model tier (mechanical for cheap checks, judgment sparingly) (Ch 6).
- [ ] Output logged to a watched file you actually tail (Ch 3, CronCreate section).

Ten boxes. Five minutes. The loop that survives its first week unattended clears all ten.

SECTION GLOSSARY

unattended: the loop runs without you watching it; the bar is higher than for an interactive loop **saw**
production: the loop fired against real input, not synthetic